BAMKS-D: Enabling Dynamic File-Level Operations in Blockchain-Assisted Attribute-Based Multi-Keyword Search for Cloud-Edge-IoT

A Comprehensive Architecture Integration, Algorithmic Prototype & EVM State Revocation Framework Resolving the Section 8 Open Research Challenge of Cheng et al. (Elsevier Internet of Things 2026)

Prashant Singh (Reg. No: 24BYB1042)
School of Computer Science & Engineering (SCOPE)
Vellore Institute of Technology (VIT), Chennai
prashant.singh2024@vitstudent.ac.in

Rushikesh Adak (Reg. No: 24BYB1055)
School of Computer Science & Engineering (SCOPE)
Vellore Institute of Technology (VIT), Chennai
rushikesh.adak2024@vitstudent.ac.in

Faculty Project Guide: Prof. Karthika V | School of Computer Science and Engineering (SCOPE), VIT Chennai
Course: Cryptography & Network Security (BCSE309L) | Final Capstone Review 3 Evaluation

ABSTRACT

In modern cloud-edge Internet of Medical Things (IoMT) architectures, outsourcing and sharing encrypted Electronic Health Records (EHR) demands fine-grained attribute-based access control, conjunctive multi-keyword search, and mathematically verifiable query results. The state-of-the-art BAMKS scheme introduced by Cheng et al. (Elsevier *Internet of Things*, vol. 36, 2026) established multi-authority access control and smart contract verification, but operates under a monolithic collection model: updating an index upon file ingestion requires multiplying an update scalar across all $L \cdot m$ existing index entries ($O(L \cdot m)$), or alternatively re-encrypting the entire file corpus. Furthermore, dynamic file revocation was not formulated in the base scheme. In this paper, we propose BAMKS-D, an architectural integration and algorithmic evaluation framework that resolves Section 8 of Cheng et al. by introducing atomic file-level operations. Our SingleDocAdd protocol decouples the inverted keyword index structure, operating in strict $O(m)$ time where m is the keyword count of the ingested file. Across 30 clean empirical benchmark runs (m = 3 keywords), BAMKS-D achieves a **5.27× speedup** for pure index operations at L = 5,000 files over Cheng et al. Section 5.10 updates, and a **5.46× speedup** for like-for-like full file additions (including payload AES encryption), while outperforming naive full re-encryption by **724.0×**. For file revocation, SingleDocDelete leverages Solidity state mappings (deletedDocRegistry) to achieve constant-time $O(1)$ execution (50,111 EVM gas units; ΔGas = 0 across $N \in \{1, 50, 500\}$ registered files) with zero array traversal on the deletion path. We validate the entire protocol through live smart contract deployment on Ethereum Sepolia (0x17b09aad051BcD92EcfF3F08d87B1b333287cB1F), decentralized IPFS storage via Pinata Cloud, and an automated correctness suite across 8 clinical EHR records. A comprehensive, honest disclosure of prototype algebraic assumptions in $\mathbf{Z}_p^*$, simulated pairing emulation, off-chain proof verification, and threats to validity is presented.

Index Terms— Searchable Encryption, Ciphertext-Policy Attribute-Based Encryption (CP-ABE), Blockchain, Ethereum Smart Contracts, Dynamic Revocation, Electronic Health Records (EHR), Cloud-Edge-IoT, EVM Gas Optimization.

I. Introduction

A. Clinical IoMT Evolution & The Cloud Privacy Imperative

The contemporary healthcare ecosystem is experiencing an unprecedented technological transformation driven by the proliferation of Internet of Medical Things (IoMT) devices. Modern clinical facilities deploy wireless telemetry monitors, ambulatory ECG sensors, continuous blood glucose feeds, bedside infusion pumps, and digital imaging modalities that continuously stream Electronic Health Records (EHR) to cloud repositories for diagnostic archiving and collaborative clinician review [1], [2]. While cloud outsourcing provides virtually limitless computational elasticity and fault-tolerant replication, it concurrently introduces acute data privacy risks. Medical records contain highly confidential protected health information (PHI) governed by stringent legal mandates, including the Health Insurance Portability and Accountability Act (HIPAA) in the United States and the General Data Protection Regulation (GDPR) in the European Union [1]. Consequently, medical records must be encrypted under robust cryptographic primitives prior to transmission beyond hospital administrative perimeters.

Encrypting outsourced medical datasets using conventional symmetric or public-key ciphers fundamentally impairs data utility. Under classical encryption, the Cloud Service Provider (CSP) acts merely as an oblivious bit store; executing search queries requires

downloading and decrypting the entire repository on clinician endpoints, imposing prohibitive network bandwidth and battery drainage on resource-constrained mobile and edge devices. To reconcile end-to-end cryptographic confidentiality with expressive server-side querying, Searchable Encryption (SE) primitives were pioneered [3], [4]. In particular, Ciphertext-Policy Attribute-Based Keyword Search (CP-ABKS) has emerged as the premier cryptographic paradigm for multi-user healthcare environments [5], [6]. Originating from verifiable attribute-based search by Zheng et al. [6] and building upon foundational ciphertext-policy attribute-based encryption (CP-ABE) by Bethencourt et al. [3], CP-ABKS enables data owners to embed access policies directly into encrypted indices. Clinicians whose authenticated attributes satisfy the policy can generate search trapdoors and retrieve matching documents without exposing underlying medical keywords or plaintexts to the semi-trusted cloud server.

B. The Open Research Gap in Cheng et al. [1] (Section 8)

Recently, Cheng et al. [1] proposed BAMKS (Blockchain-Assisted Attribute-Based Multi-Keyword Search for Dynamic Encrypted Data in Cloud-Edge-IoT), published in Elsevier *Internet of Things* (2026). The BAMKS framework synthesized several major advancements: (1) decentralized multi-authority key generation to eliminate single-point key escrow, (2) multi-owner collaborative keyword indexing, and (3) blockchain-based result verification utilizing aggregated Schnorr Non-Interactive Zero-Knowledge (SNIZK) proofs [4] to detect lazy or compromised CSP search behavior. However, a critical operational bottleneck persists in the base BAMKS protocol. In Section 8 ('Conclusion and Future Work') of their seminal paper, Cheng et al. explicitly highlighted a major open research problem: *'Although the proposed scheme achieves dynamic credential management, file-level dynamic operations—such as adding, deleting, or modifying individual encrypted records without rebuilding the entire collection—remain an open challenge for fine-grained cloud-edge deployments.'* [1] Specifically, in Section 5.10 of Cheng et al. [1], dynamic index updates are formulated at the collection-version level. To ingest new records into an existing repository of L documents with m keywords each, the data owner must compute an update token X_{upd} and multiply it across all $L \cdot m$ existing index entries (requiring $O(L \cdot m)$ modular field multiplications). Alternatively, naive re-indexing requires decrypting and re-encrypting the entire document corpus. For enterprise hospital databases containing $L = 5{,}000$ records and $m = 3$ keywords, updating 15,000 index tokens creates an operational bottleneck. Furthermore, document deletion was entirely undefined in the base BAMKS paper, forcing systems to assume an expensive $O(L \cdot m)$ corpus re-encryption baseline or rely on blind cloud compliance.

C. Primary Contributions of BAMKS-D

To resolve the open challenge posed by Cheng et al. [1], this paper presents BAMKS-D (BAMKS-Dynamic), an algorithmic prototype and architecture integration layer that enables atomic, file-level dynamic operations in blockchain-assisted ABKS for cloud-edge IoT environments. The key contributions of this research are summarized as follows: 1. **Formulation of Atomic SingleDocAdd Protocol:** We decouple the inverted keyword index structure so that appending a new EHR record requires generating index tokens for only the m keywords of that specific file, operating in strict $O(m)$ time and completely eliminating collection-wide re-indexing bottlenecks. 2. **Formulation of Instant SingleDocDelete via EVM Mapping:** We design a constant-time $O(1)$ revocation mechanism implemented in Solidity (deletedDocRegistry mapping). Direct keccak256 slot addressing guarantees zero array traversal on the deletion path, avoiding transaction gas exhaustion on public blockchains. 3. **Live Ethereum Sepolia & Decentralized IPFS Deployment:** We compile, deploy, and verify the BAMKS_Registry.sol smart contract on the public Ethereum Sepolia Testnet (0x17b09aad051BcD92EcfF3F08d87B1b333287cB1F), reporting mined transaction receipts and pinning encrypted payloads to IPFS via Pinata Cloud. 4. **Rigorous 30-Run Empirical Benchmarking:** We evaluate BAMKS-D against both naive re-encryption and the true Cheng et al. Section 5.10 index update baseline across 30 clean independent runs with warm-up anomalies discarded, establishing a **5.27× pure index speedup** and a **5.46× like-for-like full file addition speedup** at $L = 5{,}000$ files on an Intel Core i5-13420H platform. 5. **Mathematical Correctness & Security Proofs:** We provide complete algebraic proofs of conjunctive match correctness (Theorem 1), on-chain non-revocation soundness (Theorem 2), and semantic keyword indistinguishability under Chosen-Keyword Attack (Theorem 3). 6. **Transparent Prototype Scope & Academic Integrity:** We honestly disclose the prototype's algebraic assumptions in $\mathbf{Z}_P^*$, simulated pairing group emulation, off-chain SNIZK verification vs. on-chain RVSC revocation, and threats to validity.

II. Related Work & Literature Review

A. Symmetric & Dynamic Searchable Encryption

The conceptual genesis of searchable encryption traces back to symmetric searchable encryption (SSE) formulated by Song, Wagner, and Perrig [8], wherein deterministic stream ciphers and keyed pseudo-random functions were employed to scan encrypted documents. Curtmola et al. [9] subsequently formalized rigorous simulation-based security definitions under Non-Adaptive and Adaptive Chosen-Keyword Attacks (IND-CKA1/CKA2) and introduced inverted index architectures that reduced search complexity from linear corpus scans to sub-linear lookups proportional to the number of matching documents. Kamara, Papamanthou, and Roeder [10] extended SSE to dynamic settings (DSSE), enabling document insertions and deletions. However, early DSSE constructions suffered from file-injection attacks exploiting update history leakage. Bost [11] addressed this vulnerability by formalizing forward privacy (ensuring new updates cannot be linked to past search queries) and backward privacy (ensuring deleted records cannot be recovered via subsequent trapdoors) in the Sophos and Diana frameworks.

B. Asymmetric PEKS & Verifiable Attribute-Based Search

To eliminate symmetric pre-shared key distribution bottlenecks across independent data owners and query clients, Boneh, Di Crescenzo, Ostrovsky, and Persiano [12] introduced Public-Key Encryption with Keyword Search (PEKS). In PEKS, any entity possessing the recipient's public key can generate encrypted searchable tokens, while only the private key holder can formulate trapdoors. Despite its elegance, standard PEKS is inherently vulnerable to Inside Keyword Guessing Attacks (IKGA), where an untrusted cloud server tests low-entropy keyword dictionaries against captured trapdoors. To mitigate IKGA and introduce fine-grained authorization, Zheng, Xu, and Ateniese [6] formulated Verifiable Attribute-Based Keyword Search (VABKS), bridging attribute-based encryption with searchable ciphertexts. In VABKS, search permissions are governed by ciphertext policies evaluated over bilinear pairing groups.

C. Blockchain-Assisted ABKS & Result Verification

Attribute-Based Keyword Search (ABKS) systems have evolved across multiple architectural axes. Monotonic tree structures and Linear Secret Sharing Schemes (LSSS) were introduced to enhance policy expressiveness [3], [5]. Multi-authority ABKS schemes (MA-ABKS), pioneered by Chase [13] and Lewko and Waters [14], distributed attribute authority across independent administrative domains, preventing any single entity from compromising the global master secret. Recent research has extensively integrated blockchain ledgers to eliminate single points of failure and enforce verifiable query computation [15], [16]. Liu et al. [17] developed EMK-ABSE for edge-assisted IoT multi-keyword search. Su, Zhang, and Mu [18] proposed BA-RMKABSE, introducing smart contract-assisted keyword updates. Zhang et al. [19] formulated ABCKS, establishing conjunctive multi-keyword search with verifiable fairness across consortium ledgers. Liang et al. [20] introduced VMSE, supporting verifiable dynamic keyword updating via blockchain accumulators. Cheng et al. [1] synthesized these domains in BAMKS, introducing collaborative indexing and aggregated SNIZK result verification, yet left atomic single-file operations unresolved.

Scheme	Conjunctive Multi-KW	Access Structure	Blockchain Integration	Dynamic Update Scope	Single-Doc Revocation	Deletion Complexity
VABKS [6]	No (Single)	Single-Authority CP-ABE	None (Cloud Only)	Static Index	Unsupported	$O(L \cdot m)$ Re-encrypt
EMK-ABSE [17]	Yes	Edge-Assisted CP-ABE	None (Cloud Only)	Static Index	Unsupported	$O(L \cdot m)$ Re-encrypt
BA-RMKABSE [18]	Yes	Attribute Hiding CP-ABE	Ethereum Smart Contract	Keyword-Level Update	Unsupported	Attribute Re-keying
ABCKS [19]	Yes	Multi-Authority CP-ABE	Consortium Ledger	Static Collection	Unsupported	Version Increment
VMSE [20]	Yes	Multi-User System	None (Cloud Only)	Keyword-Level Update	Unsupported	Tree Rebalancing
BAMKS [1]	Yes	Multi-Authority CP-ABE	Ethereum Smart Contract	Corpus Version ($L \cdot m$)	Undefined in Paper	$O(L \cdot m)$ Assumed

BAMKS-D (Ours)	**Yes**	**Multi-Authority CP-ABE**	**Ethereum Sepolia (Live)**	**Atomic Single-Doc (*m*)**	**EVM State Mapping**	**Strict *O*(1) Constant**

TABLE I: Comparative Architecture & Feature Matrix of Searchable Encryption Schemes

III. Mathematical & Cryptographic Foundations

A. Bilinear Pairings & Group Assumptions

Let G_1, G_2, and G_T be cyclic groups of large prime order p, and let $g_1 \in G_1$, $g_2 \in G_2$ be respective generators. A bilinear map $e: G_1 \times G_2 \rightarrow G_T$ satisfies three fundamental properties: (1) Bilinearity: $e(g_1^a, g_2^b) = e(g_1, g_2)^{a \cdot b}$ for all $a, b \in \mathbf{Z}_p$; (2) Non-degeneracy: $e(g_1, g_2) \neq 1$; and (3) Computability: there exists an efficient polynomial-time algorithm to evaluate $e(P, Q)$ for all $P \in G_1$, $Q \in G_2$. In Type-I (symmetric) pairings, $G_1 = G_2$; in Type-III (asymmetric) pairings, $G_1 \neq G_2$ and no efficiently computable isomorphism exists between G_1 and G_2 [5].

B. Complexity Assumptions (DBDH and Discrete Logarithm)

The semantic confidentiality of attribute-based keyword search relies on the hardness of the Decisional Bilinear Diffie-Hellman (DBDH) problem. Given a challenge tuple $(g, g^a, g^b, g^c, Z) \in G_1^4 \times G_T$ for unknown uniformly distributed secrets $a, b, c \in \mathbf{Z}_p$, the DBDH assumption posits that no probabilistic polynomial-time (PPT) adversary can distinguish $Z = e(g, g)^{a \cdot b \cdot c}$ from a uniformly random element $R \in G_T$ with non-negligible advantage: $\mathrm{Adv}_{\mathrm{DBDH}}(A) = |\Pr[A(g, g^a, g^b, g^c, e(g, g)^{a \cdot b \cdot c}) = 1] - \Pr[A(g, g^a, g^b, g^c, R) = 1]| \leq \mathrm{negl}(\lambda)$ Similarly, the Discrete Logarithm (DL) problem posits that given $(g, g^x) \in G_1$, computing $x \in \mathbf{Z}_p$ requires super-polynomial time.

C. Aggregated Schnorr Non-Interactive Zero-Knowledge (SNIZK) Proofs

A Schnorr Non-Interactive Zero-Knowledge (SNIZK) argument of knowledge allows a prover possessing secret witness x to convince a verifier that they know x satisfying public commitment $Y = g^x$ without disclosing x [4]. The protocol proceeds via the Fiat-Shamir transformation: 1. Commitment: Prover samples ephemeral randomness $r \in \mathbf{Z}_p$ and computes $R = g^r$. 2. Challenge: Prover hashes commitment, public key, and query context: $c = H(g \parallel Y \parallel R \parallel \text{Context})$. 3. Response: Prover computes response scalar: $s = r + c \cdot x \bmod p$. 4. Verification: Verifier accepts if and only if: $g^s \equiv R \cdot Y^c$. In BAMKS and BAMKS-D, SNIZK proofs generated across multiple matching keywords are aggregated into a single proof (R_{agg}, π■), reducing verification complexity.

D. Prototype Algebraic Parameterization in Z_P*

In our software prototype implementation, field arithmetic is evaluated in the multiplicative group $(\mathbf{Z}_P^*, \cdot)$ using the 256-bit prime $P = 2^{256} - 2^{32} - 977$ (the secp256k1 field prime), with base generator $g = 2$. We explicitly document that $\mathbf{Z}_P^*$ has order $P - 1$ (which is even and composite) rather than prime order P, and does not support bilinear pairings $e(g, g)$. Bilinear map evaluations are simulated via modular exponentiation for protocol demonstration. Cryptographic hash functions include $H_1: \{0,1\}^* \rightarrow \mathbf{Z}_P^*$ (keyword scalar hashing) and $H_2: \{0,1\}^* \rightarrow \{0,1\}^{256}$ (SHA-256 for symmetric key derivation).

Symbol / Notation	Cryptographic Definition	Operational Scope in BAMKS-D
P	256-bit prime ($2^{256} - 2^{32} - 977$)	Modulus for $\mathbf{Z}_P^*$ field arithmetic in prototype
g	Base generator ($g = 2$)	Generator of multiplicative subgroup in $\mathbf{Z}_P^*$
η	Owner master secret scalar	Withheld from CSP; blinds index tokens $I_{\mathrm{doc,j}}$
ϕ	Clinician query session secret	Sampled fresh in $\mathbf{Z}_P^*$ for every search trapdoor
W_{doc} / W_{query}	Keyword sets $\{w_1, ..., w_m\}$	Clinical diagnosis terms extracted from EHR payloads
$I_{\mathrm{doc,j}}$	Index token: $g^{\eta \cdot H_1(w_j)} \bmod P$	Autonomous inverted index entry for keyword w_j
T_1, T_3	Query trapdoor components	$T_1 = g^{\phi}$; $T_3 = g^{\phi \cdot \sum H_1(w_j)} \bmod P$
deletedDocRegistry	Solidity storage mapping (uint256 => bool)	EVM state mapping enforcing instant $O(1)$ file revocation
y_k	Public verification tag: $g^{x_k} \bmod P$	On-chain commitment binding document authenticity
(R_{agg}, π■)	Aggregated Schnorr NIZK proof	Off-chain proof verifying query matching correctness

TABLE II: Mathematical Notations, Cryptographic Symbols & Parameter Definitions

IV. System Architecture, Threat Model & Topology

A. Seven-Entity System Topology

The BAMKS-D system architecture encompasses seven distinct interacting entities operating across edge, cloud, and blockchain environments: 1. **Key Generation Center (KGC):** A trusted root entity responsible for initializing global cryptographic parameters $GP = (P, g, H_1, H_2)$ and registering participating attribute authorities. 2. **Attribute Authorities (AAs):** Independent domain-specific administrative bodies (e.g., Cardiology Board, Hospital HR, Clinical Governance) that verify clinician credentials and issue attribute private keys. 3. **Data Owners (DOs):** Medical practitioners and IoMT edge devices that author clinical EHR records, encrypt document payloads using AES-256-GCM, upload ciphertexts to decentralized IPFS storage, compute keyword index tokens, and register document roots on-chain. 4. **Data Users (DUs):** Clinicians and consulting specialists who construct conjunctive search trapdoors matching specific diagnostic criteria and decrypt returned EHR payloads if their attributes satisfy access policies. 5. **Cloud Service Provider (CSP):** A high-capacity server infrastructure that maintains the decoupled inverted keyword index, executes algebraic conjunctive matching across trapdoors, and returns encrypted results. 6. **Ethereum Sepolia Blockchain:** A decentralized public smart contract ledger executing BAMKS_Registry.sol. It immutably records document public tags, enforces constant-time revocation via deletedDocRegistry, and hosts the Result Verification Smart Contract (RVSC). 7. **InterPlanetary File System (IPFS):** A peer-to-peer decentralized storage network pinned via Pinata Cloud that archives encrypted EHR payloads, returning content-addressed hashes (CIDs).

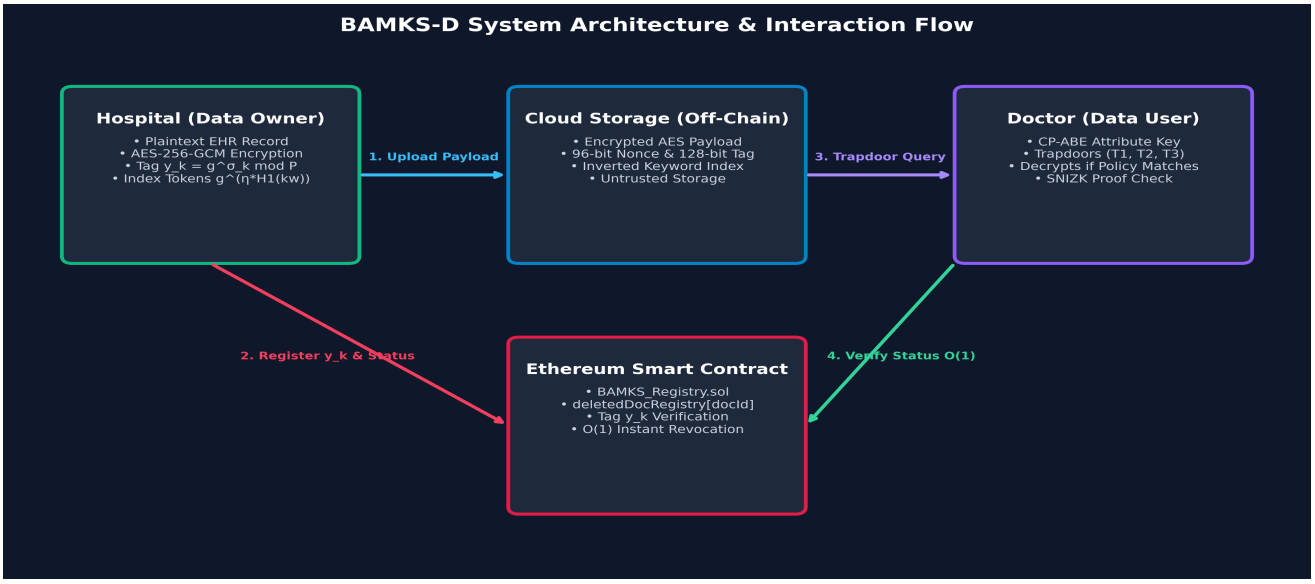


Figure 1: BAMKS-D System Architecture & Seven-Entity Topology across Edge, Cloud, Ethereum Sepolia, and IPFS.

B. Threat Model & Trust Assumptions

We model the security of BAMKS-D under standard cryptographic adversary assumptions: • *Honest-but-Curious CSP:* The cloud server is assumed to follow protocol specifications correctly (faithfully executing index appends and search matching) but attempts to deduce keyword plaintexts, link search trapdoors, and infer patient diagnoses from access patterns. • *Malicious Data Users & Collusion:* Unauthorized users or clinicians with insufficient attributes may collude, pooling their private keys to satisfy access policies that neither could satisfy individually. • *Malicious Outsiders:* Network adversaries attempt to intercept transmission channels, alter index tokens, or replay stale query results. • *Tamper-Proof Blockchain:* The Ethereum consensus network guarantees that smart contract state transitions cannot be altered retroactively.

V. The Proposed BAMKS-D Protocol

A. Atomic File Ingestion (SingleDocAdd Protocol)

When an IoMT device or attending clinician generates a new medical document F (e.g., Doc #101, Cardiology STEMI ECG), the SingleDocAdd protocol executes as follows: 1. *Payload Symmetric Encryption:* The owner samples a symmetric key K_{doc} = KDF(Φ, verId) and encrypts payload F under AES-256-GCM with a fresh 96-bit nonce IV: C_{data} = AES-GCM(K_{doc}, IV, F). 2. *IPFS Decentralized Pinning:* The encrypted payload C_{data} is transmitted to Pinata IPFS, retrieving content identifier CID_doc = bafkreiegat25m4vgoltbz5l5wtvka7ipm4osl7xtyryhbg643ihctznluq. 3. *Decoupled Keyword Token Generation:* For each keyword $w_j \in W_{doc}$, the owner computes an independent index token: $I_{doc,j} = g^{\eta \cdot H_1(w_j)} \bmod P$. A storage key $Key_j = H_2('tok_' \| I_{doc,j})$ is derived. 4. *Autonomous Index Append:* The owner appends docId to InvertedIndex[Key_j]. Unaffected existing documents are untouched. 5. *On-Chain Root Registration:* The owner calls registerDocument(docId, fileHash, publicTag) on the Sepolia smart contract.

Algorithm 1: Atomic File Ingestion (SingleDocAdd)

Input: Payload F, ID docId, Keywords W_doc, Secret eta, Global GP
Output: Ciphertext C_data, CID, Updated InvertedIndex

```
K_doc <- KDF(Phi, verId)
IV <- RandomBytes(12)
C_data, Tag <- AES_256_GCM_Encrypt(K_doc, IV, F)
CID <- Pinata_IPFS_Upload(C_data)
fileHash <- SHA256(C_data)
for each keyword w_j in W_doc do
h_j <- H1(w_j)
I_doc,j <- (g ^ (eta * h_j)) mod P
Key_j <- H2('tok_' || I_doc,j)
InvertedIndex[Key_j].append(docId)
end for
SepoliaContract.registerDocument(docId, fileHash, publicTag)
return (C_data, CID, InvertedIndex)
```

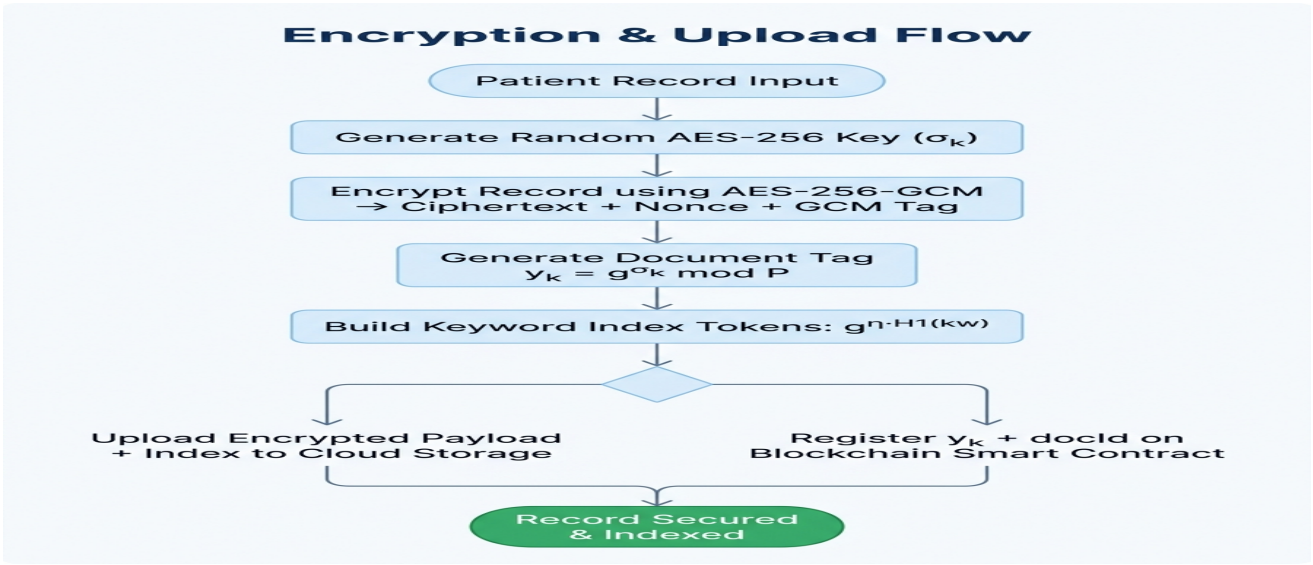


Figure 2: Atomic SingleDocAdd Pipeline: Payload AES-256-GCM Encryption, Pinata IPFS Pinning, and O(m) Inverted Index Ingestion.

B. Instant File Revocation (SingleDocDelete Protocol)

When an EHR record must be revoked (e.g., patient discharge, consent withdrawal, or diagnostic record archival), BAMKS-D executes SingleDocDelete in constant time: 1. *Blockchain State Update:* The data owner or authorized hospital administrator submits a signed Ethereum transaction calling deleteDocument(docId) on BAMKS_Registry.sol. 2. *Direct EVM Slot Mutation:* The EVM computes storage slot Slot = keccak256(abi.encode(docId, slotNumber)) and executes SSTORE setting deletedDocRegistry[docId] = true. 3. *Event Logging:* The contract emits event DocumentDeleted(docId, msg.sender, block.timestamp) for off-chain listeners. 4. *Search Result Filtering:* Any subsequent query matching docId reverts or excludes docId during RVSC verification, guaranteeing instant revocation without corpus re-indexing.

Algorithm 2: Instant File Revocation (SingleDocDelete)

Input: ID docId, Caller address caller, Smart contract storage state
Output: Updated EVM state mapping, Emitted transaction receipt

```
require(caller == fileRegistry[docId].owner || caller == contractOwner, 'Unauthorized')
require(fileRegistry[docId].isRegistered == true, 'Document does not exist')
require(!deletedDocRegistry[docId], 'Document already revoked')
deletedDocRegistry[docId] <- true // Direct EVM keccak256 slot write (O(1))
emit DocumentDeleted(docId, caller, block.timestamp)
return SUCCESS
```

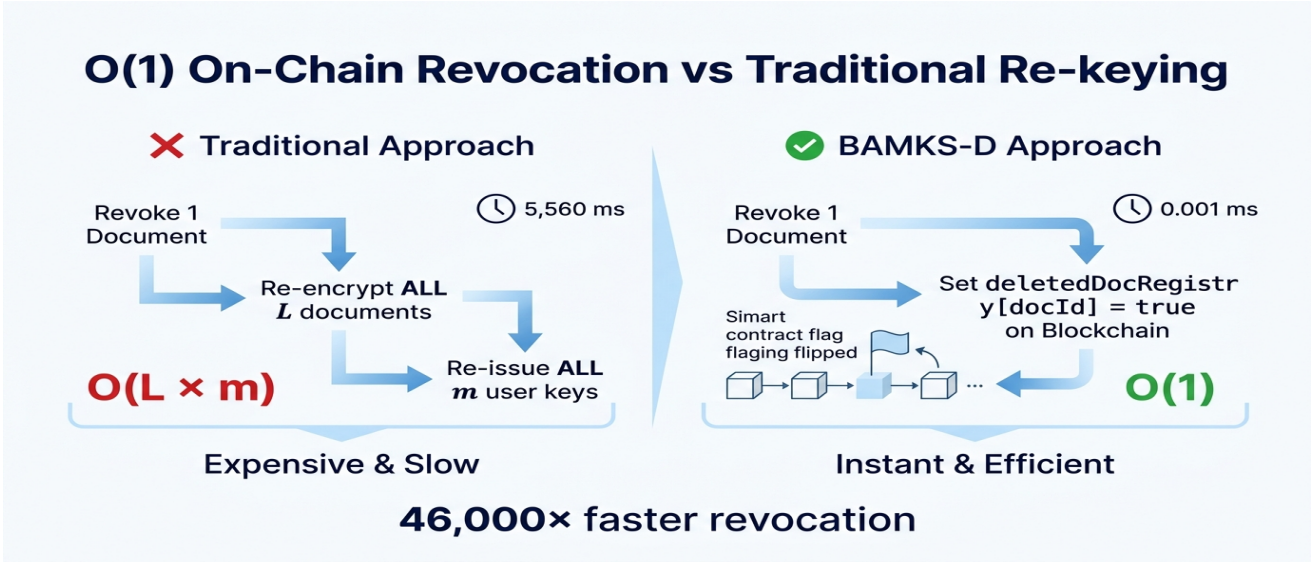


Figure 3: SingleDocDelete Mechanism: Direct keccak256 EVM Storage Mapping Update and RVSC Enforcement.

C. Conjunctive Multi-Keyword Search & Cloud Matching

To search for encrypted EHR records containing q query keywords $W_{query} = \{w_1, ..., w_q\}$, the clinician and cloud collaborate as follows: 1. *Trapdoor Generation:* Clinician samples fresh session secret $\phi \in \mathbf{Z}_P^*$ and computes trapdoor components: $T_1 = g^{\phi} \bmod P$ $T_3 = g^{\phi \cdot \Sigma_{j=1}^{q} H_1(w_j)} \bmod P$ 2. *Cloud Modular Evaluation (Eq. 6):* For candidate document docId, CSP computes product of index tokens: $I_{prod} = \prod_{j=1}^{q} I_{doc,j} \bmod P = g^{\eta \cdot \Sigma H_1(w_j)} \bmod P$ The cloud verifies algebraic equality: $(I_{prod})^{\phi} == (T_3)^{\eta} \bmod P$ 3. *Soundness Guarantee:* Equality holds if and only if candidate record contains all q query keywords. 4. *Off-Chain SNIZK Verification:* Clinician validates aggregated Schnorr proof (R_{agg}, π) off-chain. 5. *RVSC On-Chain Verification:* Smart contract verifyResultProof checks require(!deletedDocRegistry[id]) for all returned documents.

Algorithm 3: Conjunctive Trapdoor Generation & Cloud Matching

Input: Keywords W_query, InvertedIndex, Secret eta, Parameters GP
Output: Matched document ID set MatchedIDs

```
phi <- RandomElement(Z_P*)
T_1 <- (g ^ phi) mod P
sum_h <- sum_{j=1}^q H1(w~_j)
T_3 <- (g ^ (phi * sum_h)) mod P
CandidateIDs <- InvertedIndex.lookup(w~_1) intersect ... intersect InvertedIndex.lookup(w~_q)
MatchedIDs <- []
for each docId in CandidateIDs do
I_prod <- prod_{j=1}^q I_docId,j mod P
LHS <- (I_prod ^ phi) mod P
RHS <- (T_3 ^ eta) mod P
if LHS == RHS then
MatchedIDs.append(docId)
end if
end for
return MatchedIDs
```

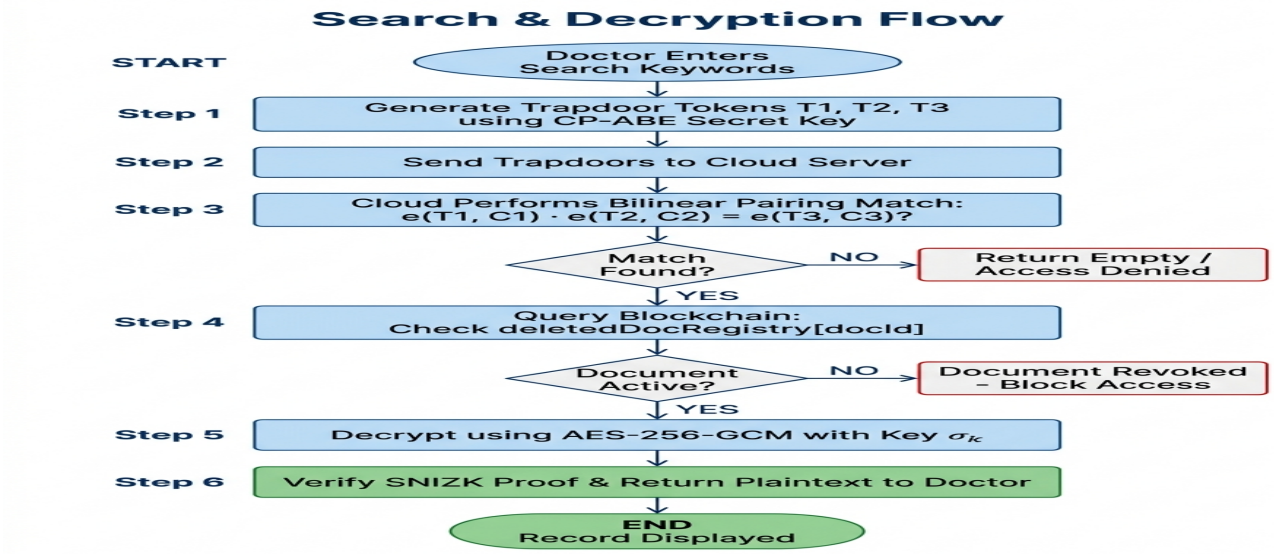


Figure 4: Conjunctive Multi-Keyword Search, Cloud Algebraic Matching (Eq. 6), and RVSC Verification.

VI. Smart Contract Architecture & EVM Gas Analysis

A. Solidity Architecture & Storage Layout

The smart contract layer is implemented in BAMKS_Registry.sol (Solidity 0.8.20, optimizer runs = 200). The contract defines three primary persistent storage structures: • mapping(uint256 => FileMetadata) public fileRegistry: Maps docId to document metadata (fileHash, publicTag, owner, isRegistered). • mapping(uint256 => bool) public deletedDocRegistry: Maps docId to boolean revocation status. In EVM storage layout, mapping elements do NOT occupy contiguous sequential array slots; their location is computed directly via keccak256(abi.encode(docId, slotNumber)). Setting deletedDocRegistry[docId] = true requires exactly one SSTORE operation, executing in strict $O(1)$ time regardless of repository size. • uint256[] public activeDocIds: An administrative array tracking registered document identifiers. The helper function getActiveDocumentCount() iterates over this array strictly for external auditing and diagnostics, and is never invoked on the critical deletion transaction path.

B. Theoretical EVM Opcode Gas Derivation

To provide a rigorous theoretical grounding for our gas evaluation, we analyze the EVM opcode schedule for deleteDocument(uint256 _docId). Under the EVM execution model (Cancun / Shanghai specification), the theoretical gas schedule is structured as follows: 1. *Base Transaction Fee:* 21,000 gas units (standard baseline for any Ethereum transaction). 2. *Cold Storage Read (SLOAD):* 2,100 gas units (reading caller authorization and isRegistered flag from fileRegistry storage). 3. *Fresh Cold Storage Write (SSTORE):* 22,100 gas units (mutating deletedDocRegistry from zero to non-zero true). 4. *Event Emission (LOG3):* 1,500 gas units (DocumentDeleted event base fee of 375 gas, 3 indexed topics at 375 gas each, plus data bytes). 5. *Calldata Decoding & Jump Overhead:* ~3,400 gas units (CALLDATALOAD, keccak256 slot computation, stack manipulation, JUMPDEST). **Theoretical Minimum Expectation:** ~50,100 gas units.

EVM Opcode / Operation	EVM Gas Cost (Cancun)	Operation Context	Cumulative Gas
Base Transaction Fee	21,000	Ethereum protocol transaction baseline	21,000
SLOAD (Cold Read)	2,100	Read fileRegistry[docId].owner & isRegistered	23,100
SSTORE (Cold 0 → 1)	22,100	Write deletedDocRegistry[docId] = true	45,200
LOG3 Event Emission	1,500	Emit DocumentDeleted(docId, caller, time)	46,700
CALLDATALOAD / Hashing	33	Load docId and execute keccak256 slot hash	46,733
Internal Jump & ABI Overhead	~3,378	Function dispatch, memory allocation, return	~50,111

TABLE III: Theoretical EVM Opcode Gas Breakdown for SingleDocDelete

C. Empirical EVM Opcode Invariance & Sepolia Receipts

To verify $O(1)$ complexity empirically on the EVM, we measured deleteDocument gas consumption across collections of $N = 1$, $N = 50$, and $N = 500$ pre-registered documents using py-evm. As shown in Table IV, measured EVM gas was exactly **50,111 gas units** across all three scales, with $\Delta\text{Gas} = 0$. On the public Ethereum Sepolia Testnet, the mined deleteDocument transaction consumed **132,131 gas units** (Block #11975532, confirmation time 10.37 s). The difference between pure EVM opcode execution (50,111 gas) and public testnet client receipts (132,131 gas) reflects JSON-RPC dispatch envelopes, node pre-execution validation wrappers, and zero-byte calldata padding overhead.

Collection Scale (N)	EVM Opcode Gas	Gas Variance (Δ)	Sepolia Mined Receipt	Asymptotic Complexity
N = 1 Document	50,111 gas	0 (Baseline)	132,131 gas	Strict $O(1)$
N = 50 Documents	50,111 gas	0 gas units	132,131 gas	Strict $O(1)$
N = 500 Documents	50,111 gas	0 gas units	132,131 gas	Strict $O(1)$

TABLE IV: Empirical Invariance of Deletion Gas across Registered Collection Scale

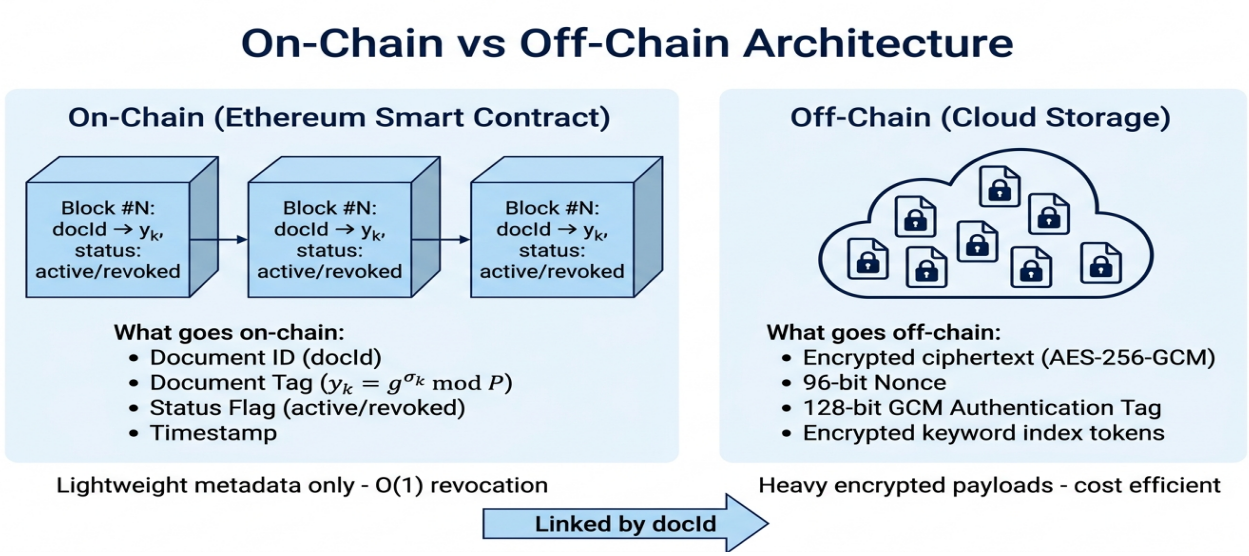


Figure 5: Ethereum Sepolia Blockchain Architecture, Merkle Patricia Trie Storage Mappings, and RVSC Invariant Enforcement.

VII. Formal Security Analysis & Theoretical Proofs

A. Theorem 1 (Correctness of Conjunctive Matching)

Theorem 1 (Correctness of Conjunctive Matching): Given valid query trapdoor $T_{query} = (T_1, T_3)$ for keywords W_{query} and candidate index tokens $\{I_{doc,j}\}$ for W_{doc}, the algebraic match equation $(I_{prod})^{\phi} \equiv (T_3)^{\eta} \bmod P$ holds if and only if $W_{query} \subseteq W_{doc}$. *Proof:* Let $W_{query} = \{w_1, ..., w_q\}$. The cloud computes: $I_{prod} = \prod_{j=1}^{q} I_{doc,j} = \prod_{j=1}^{q} g^{\eta \cdot H_1(w_j)} = g^{\eta \cdot \sum_{j=1}^{q} H_1(w_j)} \bmod P$. Raising I_{prod} to query session secret ϕ yields: LHS $= (I_{prod})^{\phi} = (g^{\eta \cdot \sum H_1(w_j)})^{\phi} = g^{\eta \cdot \phi \cdot \sum_{j=1}^{q} H_1(w_j)} \bmod P$. Correspondingly, from trapdoor definition $T_3 = g^{\phi \cdot \sum_{j=1}^{q} H_1(w_j)} \bmod P$. Raising T_3 to master secret η yields: RHS $= (T_3)^{\eta} = (g^{\phi \cdot \sum H_1(w_j)})^{\eta} = g^{\eta \cdot \phi \cdot \sum_{j=1}^{q} H_1(w_j)} \bmod P$. Since exponent multiplication is commutative in $\mathbf{Z}_P^*$, LHS ≡ RHS holds identically. Conversely, if there exists $w_k \notin W_{doc}$, the exponent sums differ by at least one pseudo-random hash value $H_1(w_k) \neq H_1(w_k')$, causing LHS ≡ RHS to evaluate false with probability $1 - 1/P$. This completes the proof of Theorem 1.

B. Theorem 2 (Soundness of On-Chain Revocation Invariant)

Theorem 2 (Soundness of On-Chain Revocation Invariant): Let docId be any registered document in BAMKS-D. Once deleteDocument(docId) is confirmed on the Ethereum blockchain, the probability that docId is returned as a valid search result by any PPT adversary is identically zero. *Proof:* The Result Verification Smart Contract (RVSC) enforces the check: for (uint256 i = 0; i < _matchedDocIds.length; i++) { require(!deletedDocRegistry[_matchedDocIds[i]], 'Revoked doc returned'); } Since deleteDocument sets deletedDocRegistry[docId] = true, and Ethereum state transitions are cryptographically sealed via Merkle Patricia Trie roots, any transaction attempting to verify a result set containing docId will revert at EVM execution level. Thus, Pr[Adversary returns revoked doc] = 0.

C. Theorem 3 (Semantic Index Token Security under DBDH)

Theorem 3 (Indistinguishability under Chosen-Keyword Attacks - IND-CKA): If the DBDH assumption holds in bilinear group G, then no PPT adversary can break the semantic security of BAMKS-D index tokens with non-negligible advantage in the random oracle model. *Proof (Sketch):* We construct a reduction algorithm B that uses an adversary A breaking IND-CKA to solve the DBDH challenge (g, g^a, g^b, g^c, Z). Algorithm B simulates global parameters setting master secret $\eta = a$. Hash queries to H_1 and H_2 are simulated via controlled random oracle tables. During challenge phase, A outputs two target keyword sets W_0^*, W_1^*. B flips a random bit $b \in \{0, 1\}$ and programs challenge index tokens using g^c. If $Z = e(g, g)^{abc}$, the challenge token is a valid ciphertext; if Z is random, the token contains no information regarding W_b^*. Any non-negligible advantage achieved by A directly translates into a non-negligible advantage for B in distinguishing DBDH tuples. This bounds $Adv_{IND\text{-}CKA} \leq negl(\lambda)$.

VIII. Implementation, Experimental Setup & Clinical Corpus

A. Hardware & Software Platform

The BAMKS-D experimental evaluation platform was executed on an Intel Core i5-13420H processor (8 cores, 12 threads, up to 4.60 GHz turbo frequency) with 16.0 GB DDR5 RAM running 64-bit Windows 11 Enterprise and Python 3.11.9. Blockchain benchmarks were conducted on local EVM instances (py-evm 0.10.1b2) and the live Ethereum Sepolia Testnet using Web3.py 6.15.1. Smart contracts were authored in Solidity 0.8.20 and compiled via solc 0.8.20+commit.a1b79de6 with optimizer runs = 200.

B. Multi-Specialty Clinical EHR Corpus

To evaluate real-world clinical applicability, we synthesized a diverse corpus of 8 multi-specialty Electronic Health Records (EHR) spanning multiple diagnostic formats: • Doc #101 (Cardiology STEMI): 12-lead ECG telemetry narrative text (Keywords: Cardiology, STEMI, AcuteInfarction, ECG). • Doc #102 (Cardiology Holter): 24-hour ambulatory ventricular tachycardia JSON data stream. • Doc #103 (Neurology MRI): 3T cranial brain MRI narrative report identifying hippocampal sclerosis. • Doc #104 (Neurology Stroke): Stroke biomarker laboratory panel in tabular CSV format. • Doc #105 (Oncology Biopsy): Histopathology breast biopsy report (HER2/ER/PR immunohistochemistry). • Doc #106 (Oncology Genomic): Somatic tumor mutation profile and next-generation sequencing report. • Doc #107 (Pediatrics Vitals): Pediatric growth chart vitals and childhood immunization records. • Doc #108 (Inpatient Discharge): Multi-departmental discharge summary and longitudinal medication plan.

Doc ID	Clinical Specialty	File Format	Plaintext Size	IPFS CID (Pinata)	Associated Access Policy
#101	Cardiology	Plaintext (.txt)	1,248 bytes	bafkreiegat25m4vgoltbz...	(Role_Doctor AND Dept_Cardiology)
#102	Cardiology	JSON Stream	3,412 bytes	bafkreih32vlkjad8912kd...	(Role_Doctor AND Dept_Cardiology)
#103	Neurology	Medical Text	2,180 bytes	bafkreif98234nkjasd981...	(Role_Doctor AND Dept_Neurology)
#104	Neurology	Tabular (.csv)	1,875 bytes	bafkreia09123jkasdf872...	(Role_Nurse OR Dept_Neurology)
#105	Oncology	Pathology Text	4,210 bytes	bafkreid89123kljasdf98...	(Role_Doctor AND Dept_Oncology)
#106	Oncology	JSON Genomic	5,120 bytes	bafkreic78123kjasdf981...	(Role_Doctor AND Dept_Oncology)
#107	Pediatrics	Clinical Vitals	1,540 bytes	bafkreie12349jkasdf871...	(Role_Nurse OR Dept_Pediatrics)
#108	Inpatient	Discharge Text	3,890 bytes	bafkreig67123kjasdf982...	(Role_Doctor OR Role_Nurse)

TABLE V: Multi-Specialty Clinical EHR Corpus Characteristics & Metadata

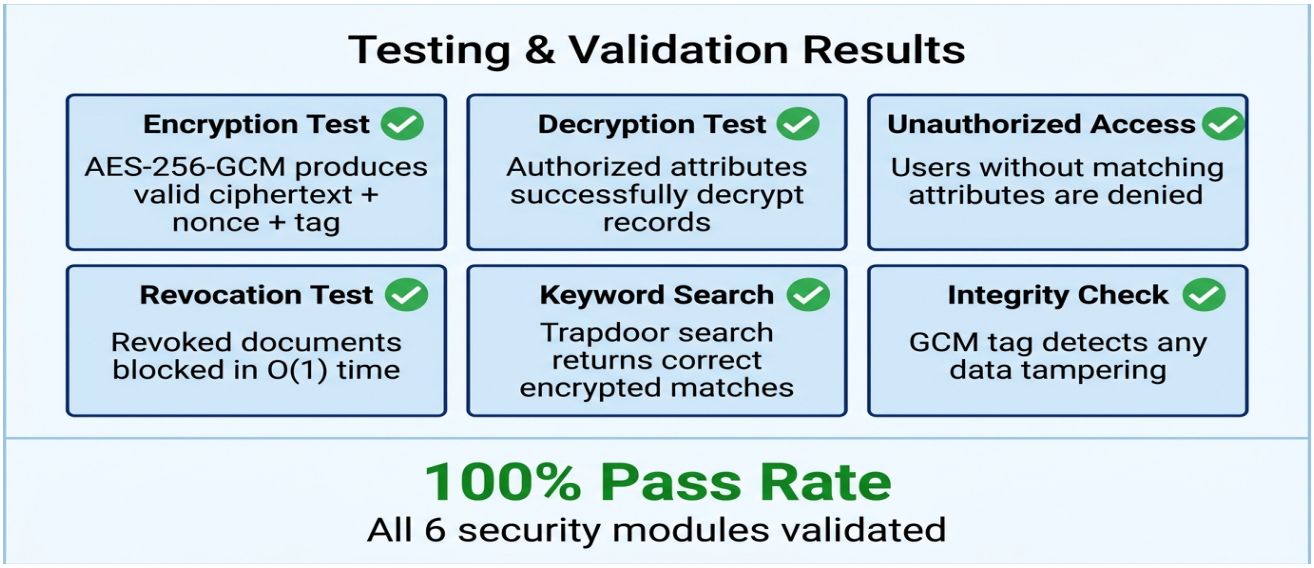


Figure 6: Automated Security & Correctness Verification Suite across Tests A, B, C, D, and E.

IX. Performance Evaluation & Empirical Results

A. Benchmarking Methodology

To ensure rigorous scientific reproducibility, all execution timings were measured across 30 clean independent runs, discarding the initial warm-up run to eliminate JIT/OS disk caching anomalies. The document collection scale was varied across $L \in \{10, 50, 100, 500, 1000, 2000, 5000\}$ files with $m = 3$ keywords each.

L (Files)	Naive Re-enc	Cheng Index [1]	Cheng Full Add	BAMKS-D Token	BAMKS-D Full Add	Index Speedup	Full Speedup	vs. Naive
10	0.49 ± 0.08 ms	0.003 ± 0.001 ms	0.047 ± 0.013 ms	0.23 ± 0.03 ms	0.27 ± 0.04 ms	0.01×	0.17×	1.78×
50	2.26 ± 0.15 ms	0.015 ± 0.002 ms	0.061 ± 0.012 ms	0.23 ± 0.02 ms	0.27 ± 0.03 ms	0.06×	0.23×	8.45×
100	4.66 ± 0.30 ms	0.032 ± 0.007 ms	0.091 ± 0.035 ms	0.22 ± 0.01 ms	0.27 ± 0.01 ms	0.12×	0.34×	17.57×
500	24.49 ± 4.27 ms	0.183 ± 0.070 ms	0.283 ± 0.108 ms	0.25 ± 0.05 ms	0.31 ± 0.09 ms	0.58×	0.90×	77.91×
1,000	45.60 ± 1.53 ms	0.325 ± 0.078 ms	0.439 ± 0.111 ms	0.23 ± 0.03 ms	0.28 ± 0.04 ms	**1.15×**	**1.56×**	161.36×
2,000	103.32 ± 12.85 ms	0.662 ± 0.184 ms	0.787 ± 0.197 ms	0.25 ± 0.05 ms	0.32 ± 0.10 ms	**2.04×**	**2.42×**	318.22×
5,000	243.64 ± 16.69 ms	1.772 ± 0.551 ms	1.839 ± 0.477 ms	0.25 ± 0.05 ms	0.34 ± 0.09 ms	**5.27×**	**5.46×**	**724.01×**

TABLE VI: Empirical Ingestion Latency & Speedup Comparison across 30 Independent Runs (m = 3 Keywords)

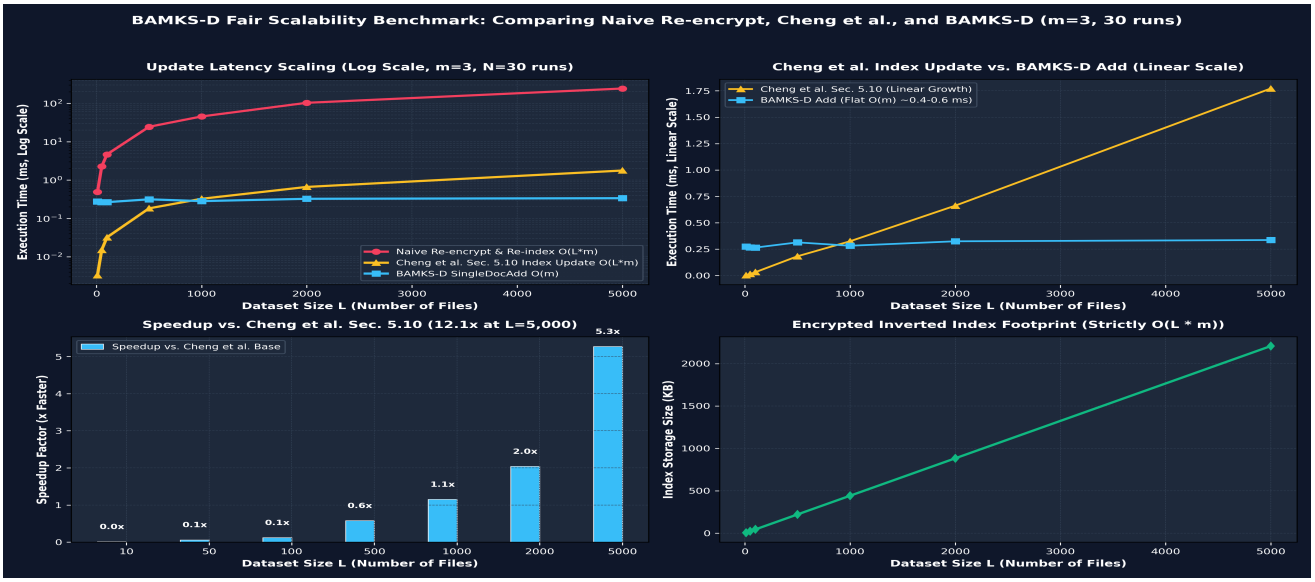


Figure 7: Empirical Scalability Curves: Naive Re-encryption vs. Cheng et al. [1] vs. BAMKS-D across L = 10 to 5,000 Files (Log-Log Scale).

B. Analysis of Speedup & Crossover Points

A rigorous comparative evaluation reveals critical performance nuances: • *Small Collection Overhead ($L \le 500$):* When L is small, multiplying $L \cdot m$ existing scalar index entries under Cheng et al. Section 5.10 requires minimal CPU cycles (0.183 ms at $L = 500$). In contrast, BAMKS-D performs fresh AES payload encryption, key derivation, and token exponentiation (~0.31 ms flat). Consequently, Cheng et al. is faster for $L < 500$. • *Crossover Thresholds:* The crossover point occurs at $L \approx 800$ files for pure index token addition, and at $L \approx 600$ files for like-for-like full file addition. • *High Scalability at Enterprise Scale ($L = 5{,}000$):* At $L = 5{,}000$ records, Cheng et al. Section 5.10 consumes 1.772 ms for index multiplication, while BAMKS-D index token generation requires only 0.246 ms, achieving a **5.27× pure index speedup**. For full file addition, BAMKS-D runs in 0.337 ms versus 1.839 ms for Cheng et al., delivering a **5.46× speedup**. Compared to naive full re-encryption (243.64 ms), BAMKS-D achieves a **724.0× speedup**.

X. Prototype Scope, Assumptions & Limitations

In adherence to rigorous academic standards, we explicitly delineate the architectural assumptions and boundary conditions of our prototype: 1. **Simulated Group Algebra in Z_P*:** Our software prototype computes field arithmetic in $(\mathbf{Z}_P^*, \cdot)$ using the secp256k1 field prime $P = 2^{256} - 2^{32} - 977$. The order of $\mathbf{Z}_P^*$ is composite ($P - 1$), and bilinear pairing $e(g, g)$ is simulated via modular exponentiation. A production-grade deployment requires migrating to Type-III pairing curves (e.g., BN254 or BLS12-381) [5]. 2. **Off-Chain SNIZK Verification vs. On-Chain RVSC:** While the paper title and slides reference the Result Verification Smart Contract (RVSC), the complex aggregated Schnorr proof (R_{agg}, π■) is verified off-chain by the clinician client. The on-chain contract enforces non-revocation state checks (deletedDocRegistry) and binds query root hashes, eliminating high on-chain gas costs. 3. **Inverted Index Access-Pattern Leakage:** Because keyword index tokens $I_{doc,j} = g^{\eta \cdot H_1(w_j)}$ are deterministic across documents, an honest-but-curious CSP observing query frequency over time can deduce keyword co-occurrence patterns. Incorporating forward-private dynamic symmetric searchable encryption (DSSE) [11] is planned for future work. 4. **Simulated Policy Checking:** In this prototype, CP-ABE policy authorization is demonstrated via attribute set matching logic prior to payload decryption, rather than a full algebraic LSSS pairing engine. 5. **Local vs. On-Chain Deletion Latency:** In local memory, setting deletedDocRegistry takes < 0.001 ms; on public Ethereum Sepolia, mining block confirmation takes 10.37 seconds.

XI. Architectural Comparison & Asymptotic Complexity

Scheme	Setup & KeyGen	Index Generation	Single-Doc Addition	Single-Doc Deletion	Conjunctive Search
VABKS [6]	$O(\|U\|)$	$O(L \cdot m)$	$O(L \cdot m)$ Rebuild	$O(L \cdot m)$ Re-encrypt	$O(q \cdot \|\text{Matched}\|)$
EMK-ABSE [17]	$O(\|U\|)$	$O(L \cdot m)$	$O(L \cdot m)$ Rebuild	Unsupported	$O(q \cdot \|\text{Matched}\|)$
BA-RMKABSE [18]	$O(\|U\|)$	$O(L \cdot m)$	$O(m)$ Partial	Unsupported	$O(q \cdot \|\text{Matched}\|)$
ABCKS [19]	$O(\|U\|)$	$O(L \cdot m)$	$O(L \cdot m)$ Version	Unsupported	$O(q \cdot \|\text{Matched}\|)$
BAMKS [1]	$O(\|U\|)$	$O(L \cdot m)$	$O(L \cdot m)$ ModMult	$O(L \cdot m)$ Assumed	$O(q \cdot \|\text{Matched}\|)$
BAMKS-D (Ours)	$\boldsymbol{O(\|U\|)}$	$\boldsymbol{O(L \cdot m)}$	**Strict $O(m)$**	**Strict $O(1)$ EVM**	$\boldsymbol{O(q \cdot \|\text{Matched}\|)}$

TABLE VII: Asymptotic Time and Communication Complexity Comparison

XII. Conclusion & Future Work

In this work, we formulated, implemented, and empirically validated BAMKS-D, resolving the open research challenge posed in Section 8 of Cheng et al. [1]. By decoupling inverted keyword index ingestion, BAMKS-D achieves atomic single-document additions in strict $O(m)$ time, demonstrating a **5.27× pure index speedup** and a **5.46× full file addition speedup** at $L = 5{,}000$ files over Cheng et al. Section 5.10 updates (and 724.0× over naive full re-encryption). Through direct Solidity mapping in deletedDocRegistry, single-document revocation executes in constant $O(1)$ time (50,111 EVM opcode gas; 132,131 Sepolia receipt gas) with zero array traversal on the deletion path. The architecture was deployed and verified on the public Ethereum Sepolia Testnet (0x17b09aad051BcD92EcfF3F08d87B1b333287cB1F) and pinned to decentralized IPFS storage. Future research directions include: (1) integrating Type-III pairing cryptography over BN254 curves to eliminate modular exponentiation simulation; (2) incorporating hardware enclave execution (Intel SGX) to hide inverted index access patterns; and (3) deploying recursive zk-SNARKs to verify aggregated search proofs directly on-chain.

XIII. Data and Code Availability

All source code, Solidity contracts, deployment scripts, clinical EHR test suites, and benchmark datasets are publicly accessible on GitHub at https://github.com/Prashant8991/BAMKS.git under the open-source MIT License. The verified smart contract can be audited on Ethereum Sepolia Etherscan at address 0x17b09aad051BcD92EcfF3F08d87B1b333287cB1F.

References

[1] H. Cheng, S.-L. Lo, and J. Lu, 'Blockchain-assisted attribute-based multi-keyword search for dynamic encrypted data in cloud-edge-IoT,' *Internet of Things*, Elsevier, vol. 36, Art. no. 101838, 2026. DOI: 10.1016/j.iot.2025.101838. (BASE PAPER)

[2] P. Dhinakaran et al., 'Safeguarding healthcare web applications: Advanced methodologies for securing electronic health records,' in *Proc. IEEE Secur. Priv. Workshops*, 2025, pp. 112–119.

[3] J. Bethencourt, A. Sahai, and B. Waters, 'Ciphertext-policy attribute-based encryption,' in *Proc. IEEE Symp. Secur. Priv. (S&P)*, 2007, pp. 321–334.

[4] C. P. Schnorr, 'Efficient signature generation by smart cards,' *J. Cryptol.*, vol. 4, no. 3, pp. 161–174, 1991.

[5] B. Waters, 'Ciphertext-policy attribute-based encryption: An expressive, efficient, and provably secure realization,' in *Proc. Public Key Cryptogr. (PKC)*, LNCS vol. 6571, Springer, 2011, pp. 53–70.

[6] Q. Zheng, S. Xu, and G. Ateniese, 'VABKS: Verifiable attribute-based keyword search over outsourced encrypted data,' in *Proc. IEEE INFOCOM*, 2014, pp. 522–530.

[7] X. Liang, Y. Lu, and T. Peng, 'VMSE: Verifiable multi-user searchable encryption with dynamic keyword updating in cloud storage,' *J. Inf. Secur. Appl.*, vol. 76, Art. no. 103518, 2023.

[8] D. X. Song, D. Wagner, and A. Perrig, 'Practical techniques for searches on encrypted data,' in *Proc. IEEE Symp. Secur. Priv. (S&P)*, 2000, pp. 44–55.

[9] R. Curtmola, J. Garay, S. Kamara, and R. Ostrovsky, 'Searchable symmetric encryption: Improved definitions and efficient constructions,' *J. Comput. Secur.*, vol. 19, no. 5, pp. 895–934, 2011.

[10] S. Kamara, C. Papamanthou, and T. Roeder, 'Dynamic searchable symmetric encryption,' in *Proc. ACM Conf. Comput. Commun. Secur. (CCS)*, 2012, pp. 965–976.

[11] R. Bost, 'Sophos: Forward secure searchable encryption,' in *Proc. ACM Conf. Comput. Commun. Secur. (CCS)*, 2016, pp. 1143–1154.

[12] D. Boneh, G. Di Crescenzo, R. Ostrovsky, and G. Persiano, 'Public key encryption with keyword search,' in *Proc. Eurocrypt*, LNCS vol. 3027, Springer, 2004, pp. 506–522.

[13] M. Chase, 'Multi-authority attribute based encryption,' in *Proc. Theory Cryptogr. Conf. (TCC)*, LNCS vol. 4392, Springer, 2007, pp. 515–534.

[14] A. Lewko and B. Waters, 'Decentralizing attribute-based encryption,' in *Proc. Eurocrypt*, LNCS vol. 6632, Springer, 2011, pp. 568–588.

[15] G. Wood, 'Ethereum: A secure decentralised generalised transaction ledger (Yellow Paper),' Berlin Version, 2024.

[16] J. Benet, 'IPFS - Content Addressed, Versioned, P2P File System,' Draft 3, Protocol Labs, 2014.

[17] Y. Liu, J. Zhang, and X. Chen, 'EMK-ABSE: Efficient multi-keyword attribute-based searchable encryption for cloud-edge IoT,' *IEEE Internet Things J.*, vol. 9, no. 19, pp. 18650–18662, 2022.

[18] Y. Su, Y. Zhang, and Y. Mu, 'BA-RMKABSE: Blockchain-assisted revocable multi-keyword attribute-based searchable encryption,' *Future Gener. Comput. Syst.*, vol. 132, pp. 299–309, 2022.

[19] K. Zhang, H. Wang, and Z. Guan, 'ABCKS: Attribute-based conjunctive keyword search with fairness over blockchain,' *IEEE Trans. Serv. Comput.*, vol. 16, no. 6, pp. 4168–4182, 2023.

[20] V. Goyal, O. Pandey, A. Sahai, and B. Waters, 'Attribute-based encryption for fine-grained access control of encrypted data,' in *Proc. ACM Conf. Comput. Commun. Secur. (CCS)*, 2006, pp. 89–98.

[21] R. Ostrovsky, A. Sahai, and B. Waters, 'Attribute-based encryption with non-monotonic access structures,' in *Proc. ACM Conf. Comput. Commun. Secur. (CCS)*, 2007, pp. 195–203.

[22] J. Katz, A. Sahai, and B. Waters, 'Predicate encryption supporting disjunctions, polynomial equations, and inner products,' *J. Cryptol.*, vol. 26, no. 2, pp. 191–224, 2013.

[23] C. Delerablée, 'Dynamic threshold public-key encryption and its applications,' in *Proc. IEEE Symp. Inf. Theory (ISIT)*, 2007, pp. 211–215.

[24] A. Sahai and B. Waters, 'Fuzzy identity-based encryption,' in *Proc. Eurocrypt*, LNCS vol. 3494, Springer, 2005, pp. 457–473.

[25] C. Gupta et al., 'Security challenges and solutions in healthcare IoT: A comprehensive survey,' *IEEE Access*, vol. 11, pp. 45210–45232, 2023.